

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

КАФЕДРА №43

ОТЧЁТ ЗАЩИЩЁН С ОЦЕНКОЙ _____

ПРЕПОДАВАТЕЛЬ

Шумова Е.О.

должность, уч. степень, звание подпись, дата инициалы, фамилия

ОТЧЕТ ПО
ЛАБОРАТОРНОЙ РАБОТЕ №5
ТЕМА: ОБРАБОТКА ИСКЛЮЧИТЕЛЬНЫХ СИТУАЦИЙ
по дисциплине: «Основы программирования»

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. № 4432Кз _____ Гиппиус К.Г.
подпись, дата инициалы, фамилия

Санкт-Петербург
2026 год

Условие

Вариант 4. В одномерном массиве, состоящем из n вещественных элементов, вычислить минимальный элемент массива и сумму элементов, расположенных между первым и последним положительными элементами. Преобразовать массив таким образом, чтобы сначала располагались все элементы, равные нулю, а потом — все остальные. Размерность массива определяется суммой двух последних цифр шифра студента; для текущего шифра используется размерность 12. В программе необходимо обработать не менее 2–3 исключительных ситуаций, одну из них — через собственный класс исключения.

Листинг программы

```
#include <algorithm>
#include <cmath>
#include <exception>
#include <iomanip>
#include <iostream>
#include <numeric>
#include <stdexcept>
#include <string>
#include <vector>

class PositiveRangeException : public std::exception {
private:
    std::string message_;

public:
    explicit PositiveRangeException(std::string message) : message_(std::move(message)) {}

    const char* what() const noexcept override {
        return message_.c_str();
    }
};

class Array {
private:
    std::vector<double> data_;
    std::size_t length_;

public:
    explicit Array(const std::vector<double>& data) : data_(data), length_(data.size()) {
        if (length_ != 12) {
            throw std::invalid_argument("Array length must be 12 for this student.");
        }
    }

    std::size_t length() const {
        return length_;
    }

    void printSourceArray() const {
        std::cout << "Array: ";
        for (double value : data_) {
            std::cout << value << ' ';
        }
        std::cout << '\n';
    }
}
```

```

double minElement() const {
    if (data_.empty()) {
        throw std::logic_error("Cannot find minimum of an empty array.");
    }

    return *std::min_element(data_.begin(), data_.end());
}

double sumBetweenFirstAndLastPositive() const {
    auto firstPositive = std::find_if(data_.begin(), data_.end(), [](double value) { return value > 0;
});
    auto lastPositive = std::find_if(data_.rbegin(), data_.rend(), [](double value) { return value >
0; });

    if (firstPositive == data_.end() || lastPositive == data_.rend()) {
        throw PositiveRangeException("The array does not contain positive elements.");
    }

    auto lastPositiveBase = lastPositive.base() - 1;
    if (firstPositive >= lastPositiveBase) {
        throw PositiveRangeException("The array must contain at least two positive elements.");
    }

    return std::accumulate(firstPositive + 1, lastPositiveBase, 0.0);
}

void moveZerosToFront() {
    std::stable_sort(data_.begin(), data_.end(), [](double left, double right) {
        return (left == 0.0 && right != 0.0);
    });
}

double at(std::size_t index) const {
    if (index >= data_.size()) {
        throw std::out_of_range("Index is outside array bounds.");
    }

    return data_[index];
}

void printTransformedArray() const {
    std::cout << "Transformed array: ";
    for (double value : data_) {
        std::cout << value << ' ';
    }
    std::cout << '\n';
}
};

int main() {
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "Lab 5, variant 4\n";

    try {
        Array validArray({0.0, -5.0, 4.5, 0.0, 7.0, -1.5, 2.0, -3.0, 0.0, 8.0, -6.0, 1.0});
        validArray.printSourceArray();
        std::cout << "Minimum element: " << validArray.minElement() << '\n';
        std::cout << "Sum between first and last positive elements: "
            << validArray.sumBetweenFirstAndLastPositive() << '\n';
        validArray.moveZerosToFront();
        validArray.printTransformedArray();
        std::cout << "Element at index 3: " << validArray.at(3) << '\n';
    } catch (const std::exception& error) {
        std::cout << "Unexpected error in valid case: " << error.what() << '\n';
    }

    try {
        Array wrongSize({1.0, 2.0, 3.0});
        wrongSize.printSourceArray();
    } catch (const std::exception& error) {
        std::cout << "Handled size exception: " << error.what() << '\n';
    }
}

```

```

    }

    try {
        Array noPositive({0.0, -1.0, -2.0, 0.0, -3.0, -4.0, 0.0, -5.0, -6.0, -7.0, -8.0, -9.0});
        std::cout << noPositive.sumBetweenFirstAndLastPositive() << '\n';
    } catch (const PositiveRangeException& error) {
        std::cout << "Handled custom exception: " << error.what() << '\n';
    }

    try {
        Array outOfRangeArray({1.0, 0.0, 2.0, 0.0, 3.0, 0.0, 4.0, 0.0, 5.0, 0.0, 6.0, 0.0});
        std::cout << outOfRangeArray.at(20) << '\n';
    } catch (const std::exception& error) {
        std::cout << "Handled out_of_range exception: " << error.what() << '\n';
    }

    return 0;
}

```

Результаты выполнения программы

```

Lab 5, variant 4
Array: 0.00 -5.00 4.50 0.00 7.00 -1.50 2.00 -3.00 0.00 8.00 -6.00 1.00
Minimum element: -6.00
Sum between first and last positive elements: 6.50
Transformed array: 0.00 0.00 0.00 -5.00 4.50 7.00 -1.50 2.00 -3.00 8.00 -6.00 1.00
Element at index 3: -5.00
Handled size exception: Array length must be 12 for this student.
Handled custom exception: The array does not contain positive elements.
Handled out_of_range exception: Index is outside array bounds.

```

Вывод

В работе реализован класс массива с обработкой стандартных и пользовательских исключений. Программа корректно выполняет вычисления по варианту и демонстрирует обработку типовых аварийных ситуаций, связанных с размером массива, отсутствием положительных элементов и выходом за границы.